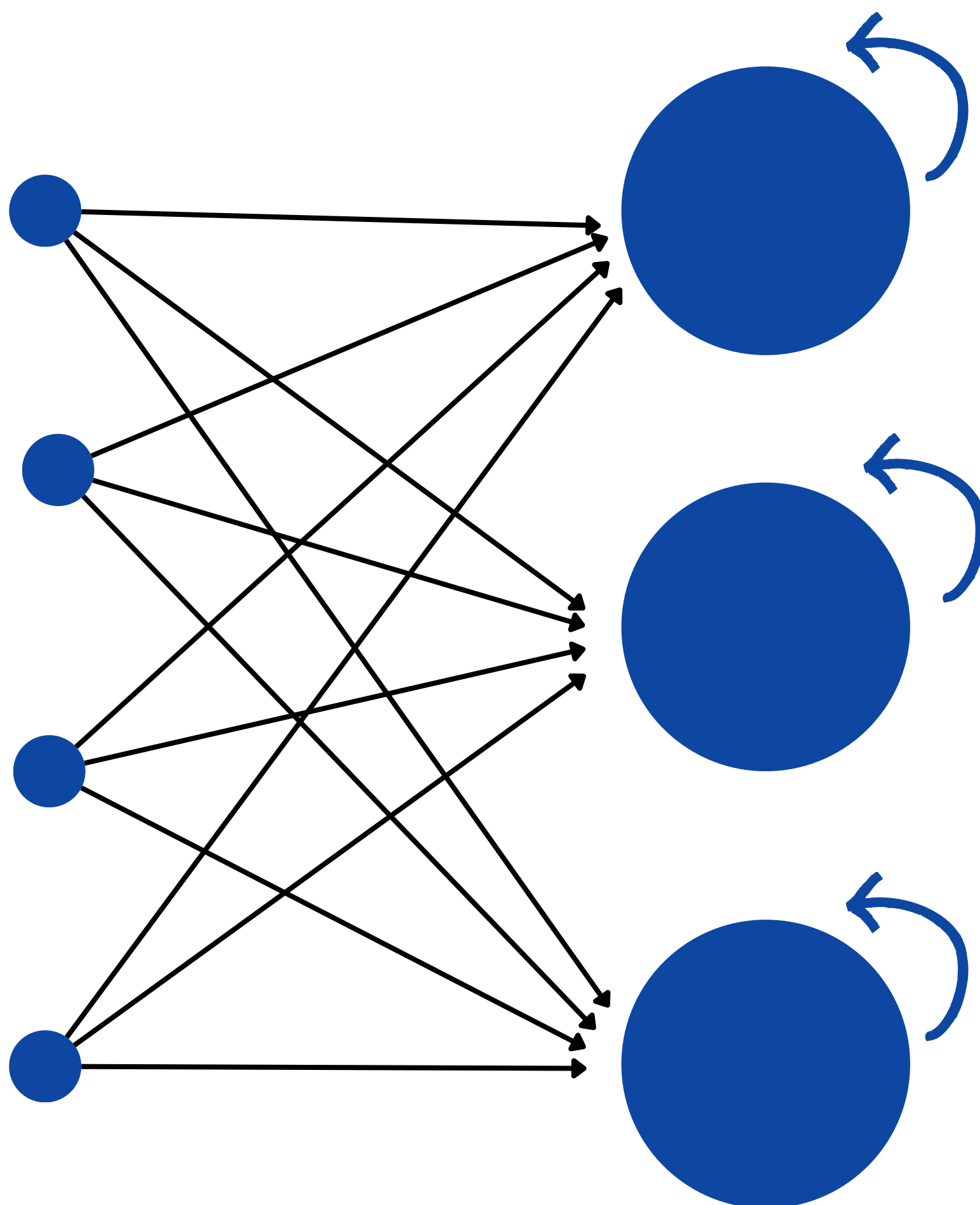


A Perfect Guide to Recurrent Neural Networks



Recurrent Neural Networks (RNNs) are a class of neural networks designed for sequential data processing. Unlike traditional feedforward networks, RNNs have connections looping back, enabling them to maintain information in their 'memory' across time steps.

Why use RNN?

- **Sequential Data:** RNNs are particularly suited for time-series data or any sequential data where previous inputs can provide context for future predictions.
- **Variable Length:** RNNs can handle variable-length input and output sequences, making them versatile for various tasks.

Advantages

- **Sequence Processing:** RNNs inherently account for sequence information, capturing patterns over time.
- **Shared Parameters:** Unlike traditional feed-forward networks, RNNs share parameters across steps, reducing the model's size.

Disadvantages

- **Vanishing & Exploding Gradients:** During backpropagation, RNNs can face issues where gradients become too small (vanish) or too large (explode), making them hard to train.
- **Limited Memory:** RNNs can struggle to remember long-term dependencies in sequences due to the vanishing gradient problem.

Python Implementation of RNN

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

# Sample data
# Assuming X_train and y_train are your training data and labels

model = Sequential()
model.add(SimpleRNN(50, activation='tanh', return_sequences=True,
                    input_shape=(X_train.shape[1], X_train.shape[2])))

model.add(SimpleRNN(50, activation='tanh'))
model.add(Dense(1))
```